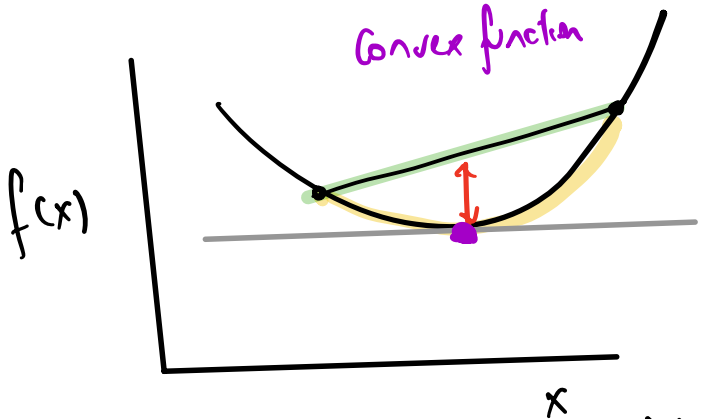
Review of ML basics: Linear regression and classification and model evaluation

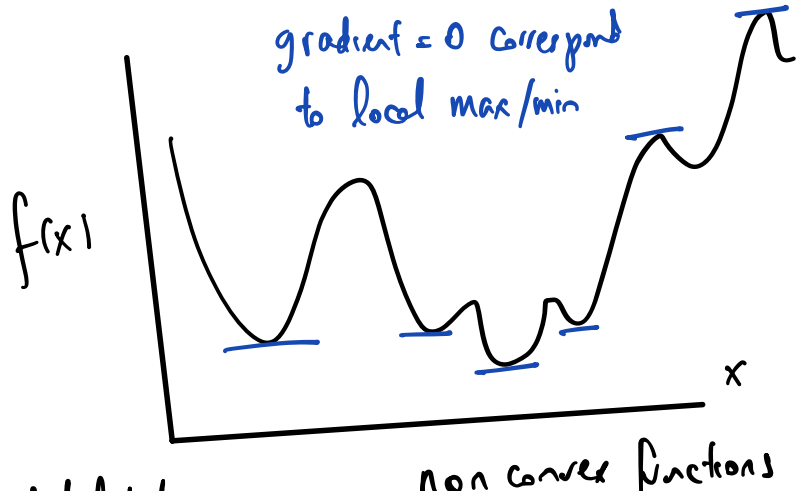
October 2nd, 2026

These slides are based on material from Professor Jonathan Kao, who also developed it based on other freely available sources including from Stanford and other resources, whom we acknowledge.

Announcements

- HW #1 is due Friday October 9, 11.59pm uploaded to Gradescope. **Note that even 1 minute late is considered a day late**
- **Please read the rules handout carefully including for late HWs etc.**
- To submit your Jupyter Notebook, print the notebook to a pdf with your solutions and plots filled in. The graders will grade your work from these notebooks. In the future, you will also have to print out code from any .py files that were modified.
- Discussion session held earlier today, Fridays 11am, will be recorded and live zoom link also is available.





non convex functions

ASIDE (not tested)

$(\underline{x}, y) \sim p(\underline{x}, y)$, $\ell(y, \hat{y})$, $\hat{y} = g(\underline{x})$

($\underline{x}$: input, y: label; y: true label, $\hat{y}$: estimated label)

$$\mathbb{E}_{(\underline{x},y)\sim p(\underline{x},y)}[\ell(y,\hat{y})] \approx \frac{1}{N}\sum_{i=1}^{N} \ell(y^{(i)}, \hat{y}^{(i)})$$

↳ empirical approximation of $\mathbb{E}[\cdot]$

Review of previous lecture

→ Supervised learning Dataset $\mathcal{D} = \{(\underline{x}^{(1)}, y^{(1)}), \ldots, (\underline{x}^{(N)}, y^{(N)})\}$ (inputs; labels)

$\hat{y}$ is close to y. $\hat{y} = g(\underline{x})$

GOAL: To find $g(\cdot)$ s.t.

- Regression: linear and polynomial versions
  - Model:

$\hat{y}$ = estimated label
$\underline{\theta}$: model parameters

$$\hat{y} = \underline{\theta}^T \hat{\underline{x}}$$

→ input data

and can be for polynomial (regression) models

$g(\underline{x}) = \underline{\theta}^T \hat{\underline{x}}$, $\hat{\underline{x}}$ conversion of $\underline{x}$ to vectorized form

  - Loss/error criterion $\ell(y^{(i)}, \hat{y}^{(i)})$

Regression

quadratic loss

$$\mathcal{L}(\underline{\theta}) = \frac{1}{2N} \sum_{i=1}^{N} |y^{(i)} - \hat{y}^{(i)}|^2, \quad \hat{y}^{(i)} = \underline{\theta}^T \hat{\underline{x}}^{(i)}$$

$y^{(i)}$ → true label; $\hat{y}^{(i)}$ → estimated label

EMPIRICAL RISK MINIMIZATION

$\mathcal{L}(\underline{\theta}) \approx \mathbb{E}[\ell(y, \hat{y})]$ → avg over data distribution

  - Optimal solution and gradients

$\|\underline{y} - \underline{\underline{X}}\underline{\theta}\|_2^2$ (vectorization)

$\rightarrow \arg\min_{\underline{\theta}} \mathcal{L}(\underline{\theta})$ by solving $\frac{\partial \mathcal{L}(\underline{\theta})}{\partial \underline{\theta}} = \underline{0}$

$$\underline{\underline{X}} = \begin{bmatrix} \underline{x}^{(1)T} \\ \vdots \\ \underline{x}^{(N)T} \end{bmatrix} \rightarrow \mathbb{R}^{N \times n}$$

$$\underline{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{bmatrix} \in \mathbb{R}^N$$

$$\hat{\underline{\theta}} = (\underline{\underline{X}}^T \underline{\underline{X}})^{-1} \underline{\underline{X}}^T \underline{y}$$

$$\nabla_{\underline{\theta}} f(\underline{\theta}) = \begin{bmatrix} \partial f / \partial \theta_1 \\ \vdots \\ \partial f / \partial \theta_n \end{bmatrix}$$

e.g., $f(\underline{\theta}) = \underline{\theta}^T \underline{x} = \underline{x}^T \underline{\theta}$

$\nabla_{\theta} f(\underline{\theta}) = \underline{x}$

$f(\underline{\theta}) = \underline{\theta}^T \underline{\underline{A}} \underline{\theta}$

$\nabla_{\theta} f(\underline{\theta}) = (\underline{\underline{A}} + \underline{\underline{A}}^T)\underline{\theta}$

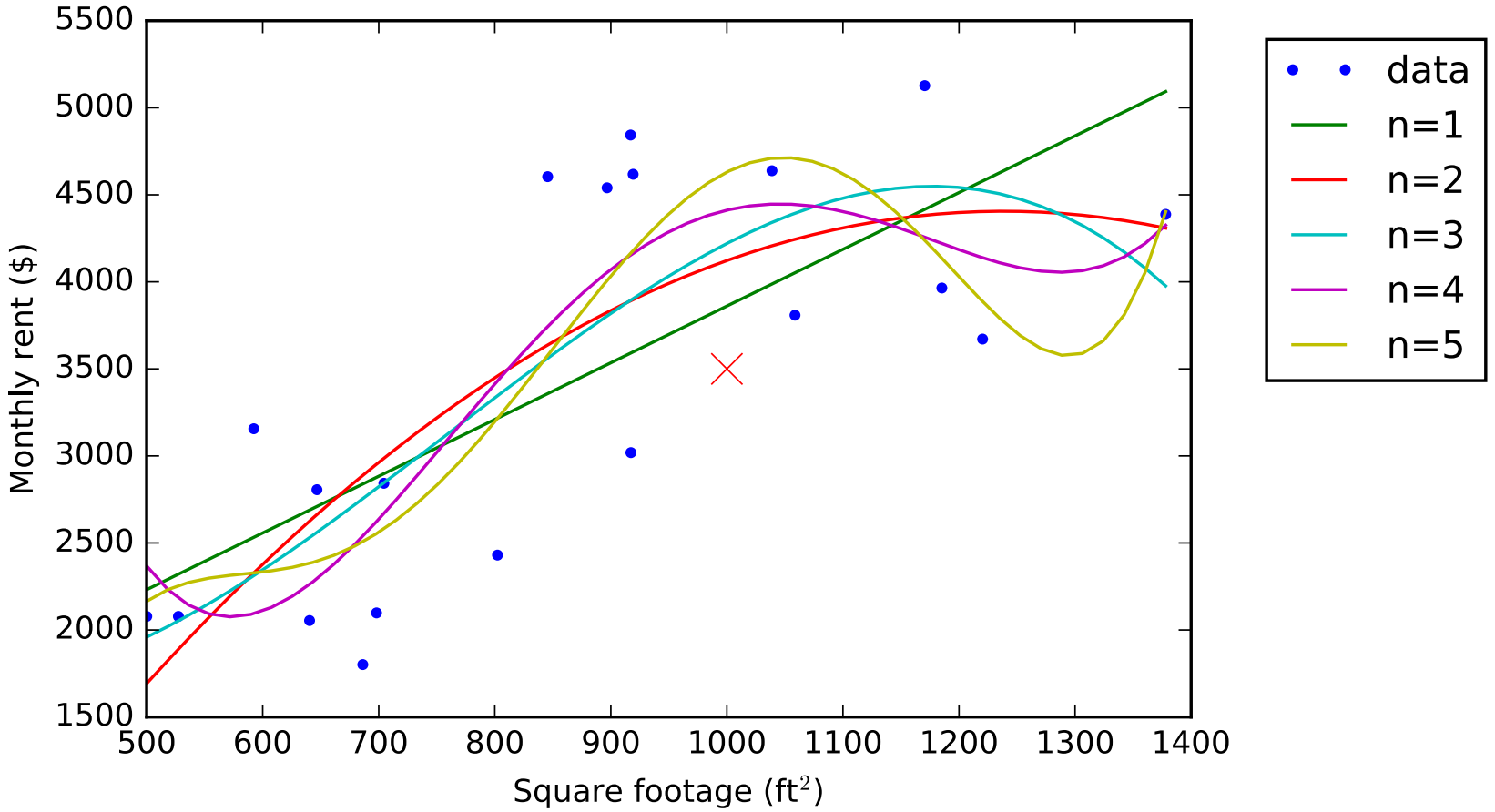
Questions

- $$\ell(\underline{\theta}) = \|\underline{y} - \underline{\underline{X}}\,\underline{\theta}\|^2 = \underline{y}^T\underline{y} + \underline{\theta}^T\underline{\underline{X}}^T\underline{\underline{X}}\,\underline{\theta} - 2\underline{y}^T\underline{\underline{X}}\,\underline{\theta}$$

$$\frac{\partial \ell}{\partial \underline{\theta}} = 0 + 2(\underline{\underline{X}}^T\underline{\underline{X}})\theta - 2\underline{\underline{X}}^T\underline{y} = \underline{0}$$

$$\Rightarrow 2(\underline{\underline{X}}^T\underline{\underline{X}})\,\underline{\theta} = 2\underline{\underline{X}}^T\underline{y} \quad \Rightarrow \quad \underline{\theta} = (\underline{\underline{X}}^T\underline{\underline{X}})^{-1}\underline{\underline{X}}^T\underline{y}$$

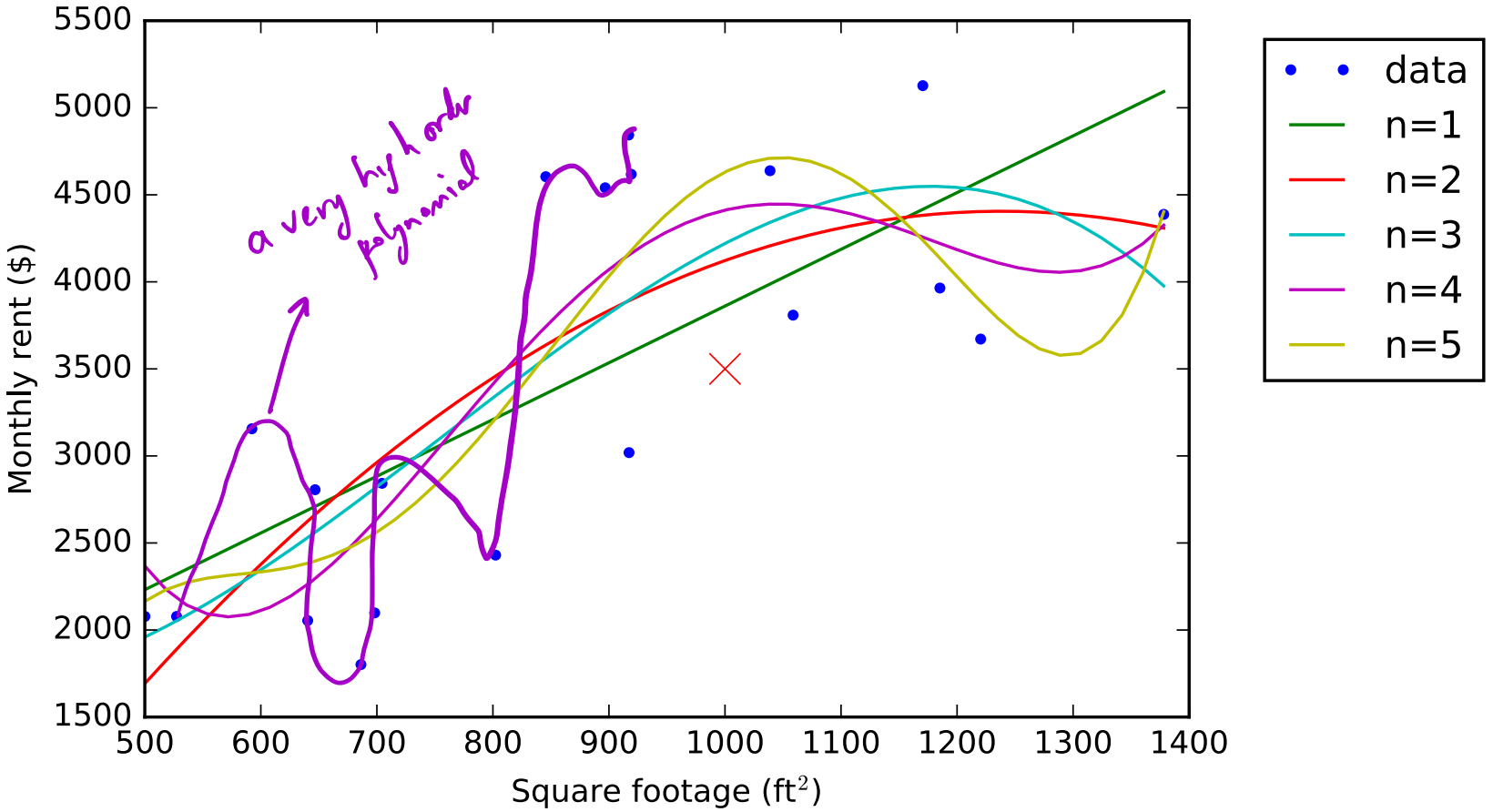
Now, let us say a new house pops up for rent…



In this scenario, the linear model best predicts the price of the new house ('x')

$$\hat{y} = b + a_1 x + a_2 x^2 + \cdots + a_n x^n \qquad \hat{y} = \underline{\theta}^T \underline{\hat{x}} \qquad \underline{\theta} = \begin{bmatrix} b \\ a_1 \\ \vdots \\ a_n \end{bmatrix}, \quad \underline{\hat{x}} = \begin{bmatrix} 1 \\ x \\ \vdots \\ x^n \end{bmatrix}$$

Model generalization



The fundamental problem here is that the more complex models may **not generalize as well** if the data come from a different model.

Question: Since training loss always reduces when we make model more complex ie in this case higher order polynomials → is that a good idea?

Training and testing data

$\hat{y} = g(\underline{x})$ → g → model learned from training

$(\underline{x}^{TEST}, y^{TEST})$, we want $\hat{y}^{TEST} = g(\underline{x}^{TEST})$ to be close to y^{TEST}

Hidden agenda: In machine learning or AI → we care about performing well on unseen data not in the training set.

$\mathcal{D} = \{(\underline{x}^{(1)}, y^{(1)}), \ldots, (\underline{x}^{(N)}, y^{(N)})\}$

Overfitting

This idea of generalization can be made more formal by introducing the concepts of a *training set* and *testing set*.

- **Training data** is data that is used to learn the parameters of your model.
- **Testing data** is data that is excluded in training and used to score your model.

There is also a notion of validation data, which we will get to later.

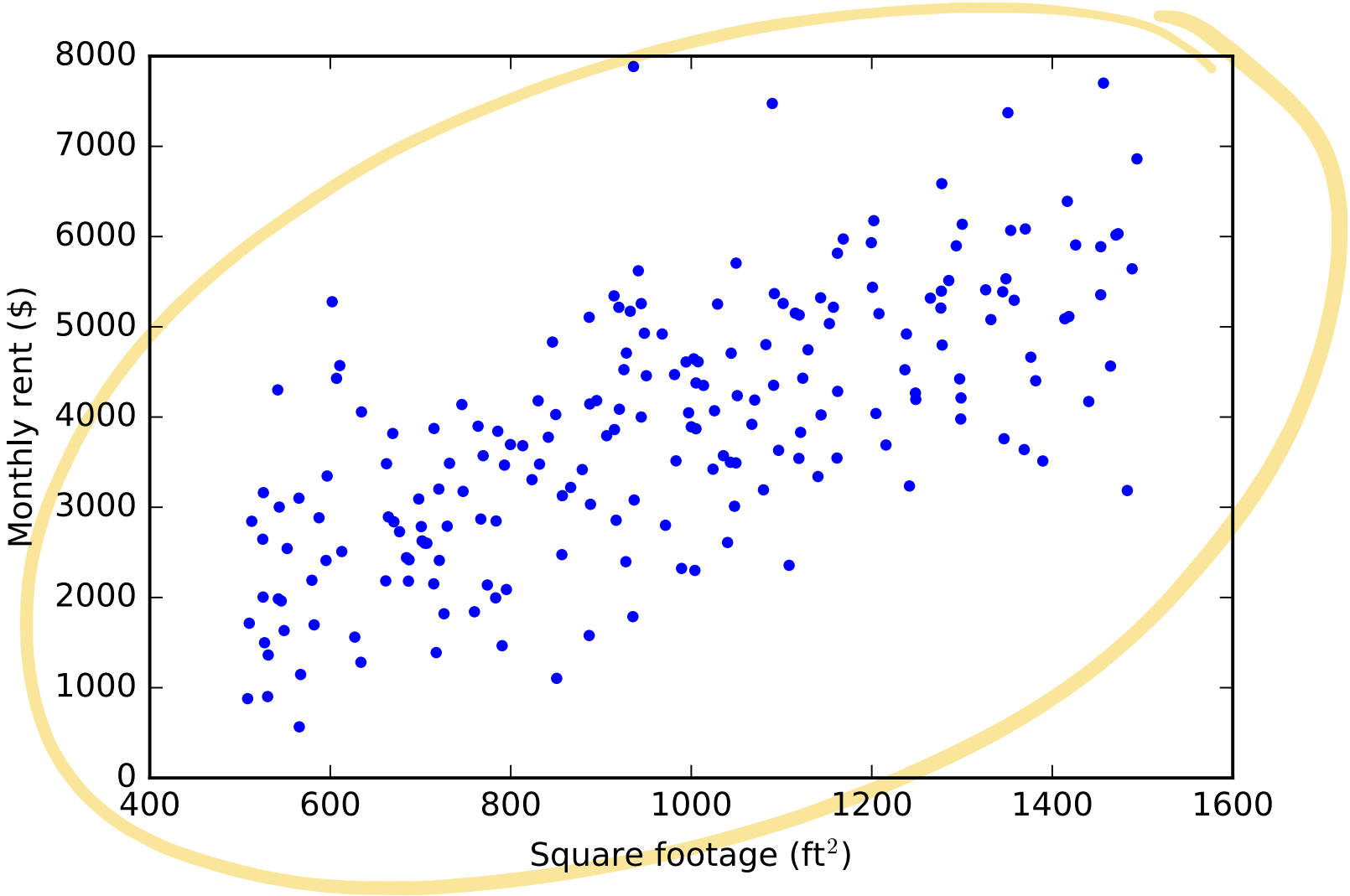
A model which has very low training error but high testing error is called *overfit*.

Underlying assumption: test data comes from "same distribution" as training data

in other words training data is representative of what you might encounter.

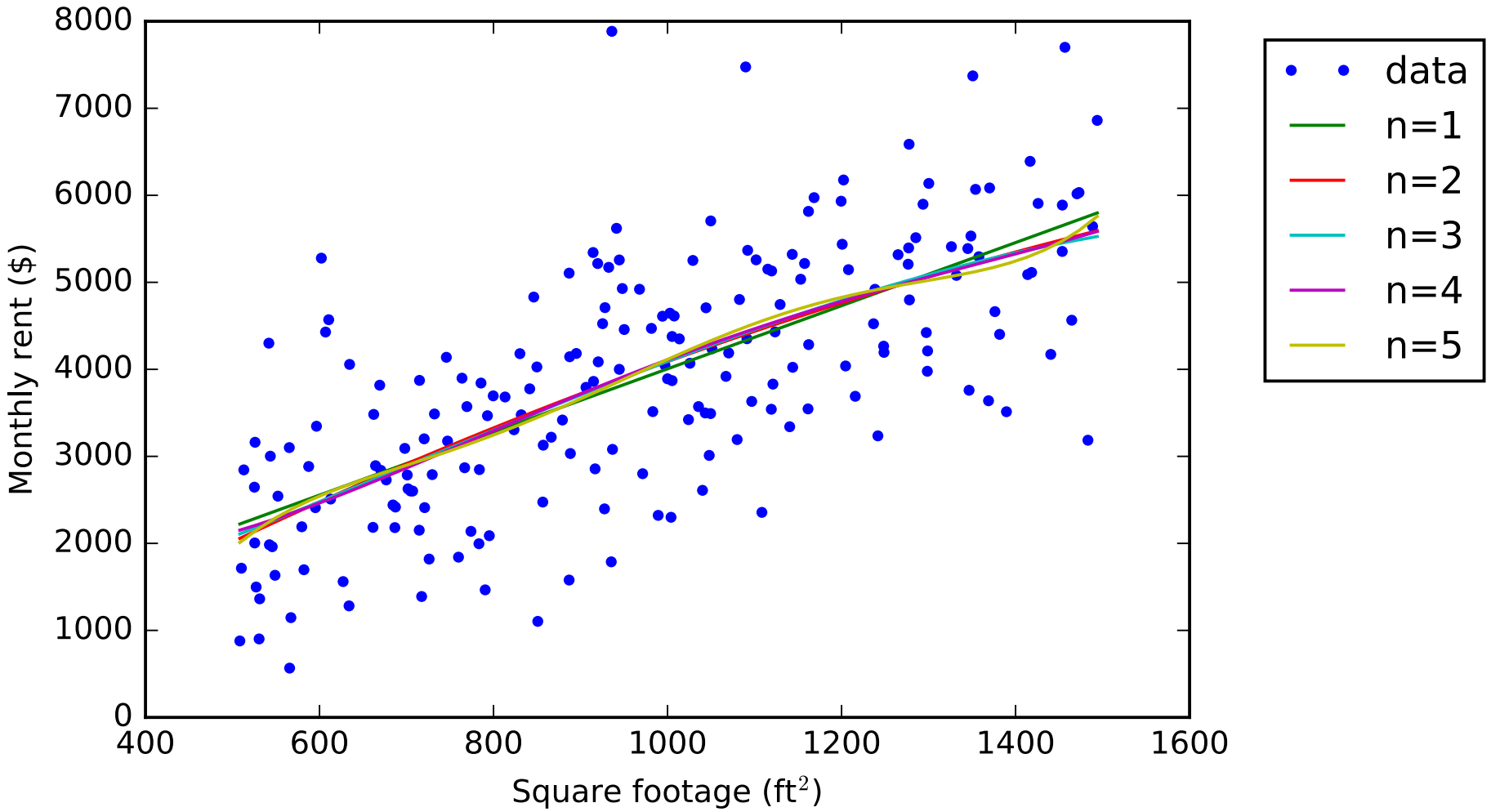
When this does not happen the terminology is "out-of-distribution" data

More data helps to avoid overfitting



More data helps to avoid overfitting

More data can admit more complex models without overfitting



More data helps to avoid overfitting

This suggests that when *a lot* of data is available, it may be appropriate to use more complex models. (Another technique we will discuss later, regularization, also helps with overfitting.) This is a shadow of things to come (i.e., neural networks which have large complexity).

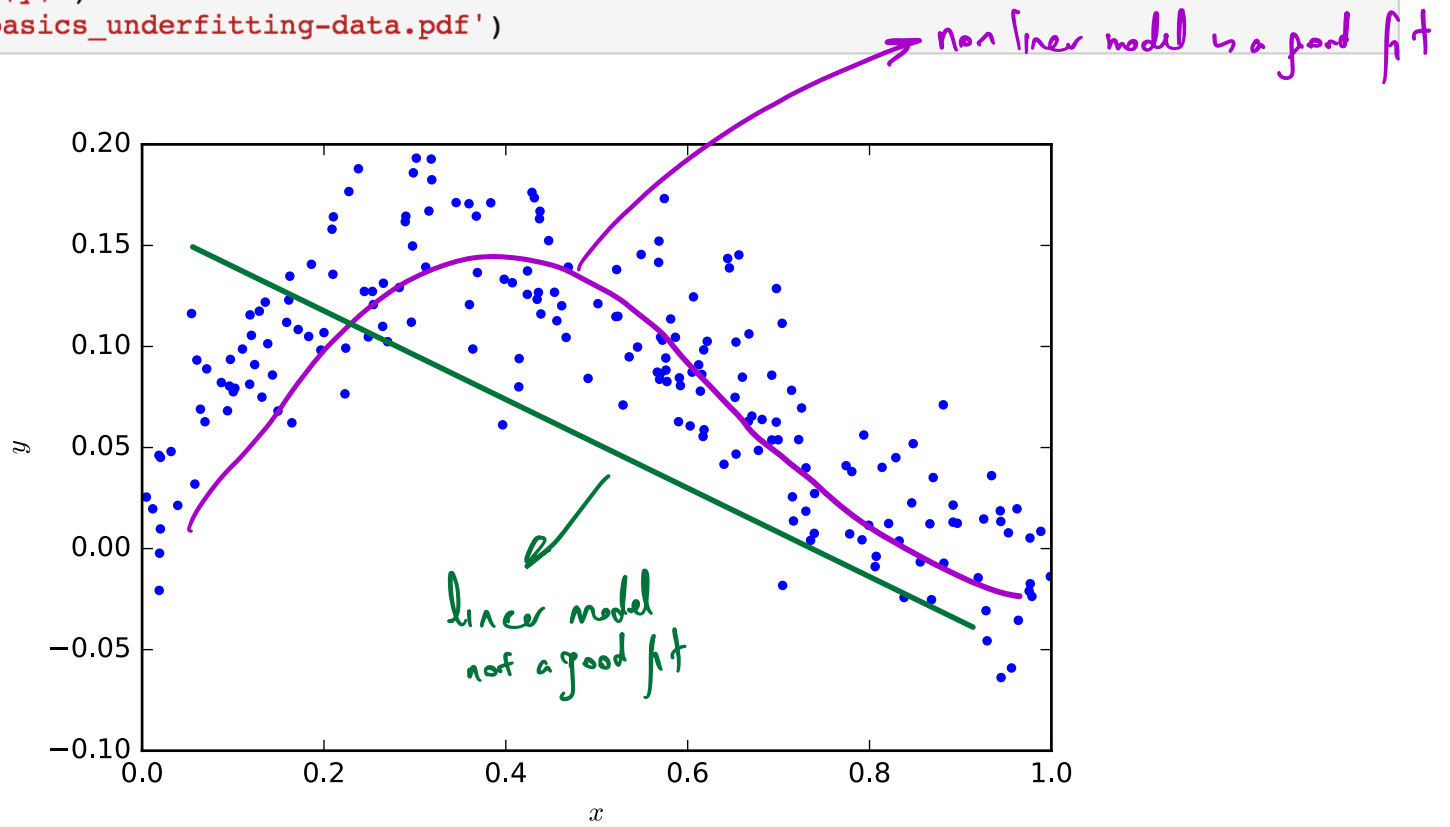
Underfitting

At the same time, we can not make the model *overly simple* as then we will underfit the data. This corresponds to a model that has both high training and high testing error, and the fit generally will not improve with more training data because the model is not expressive enough.

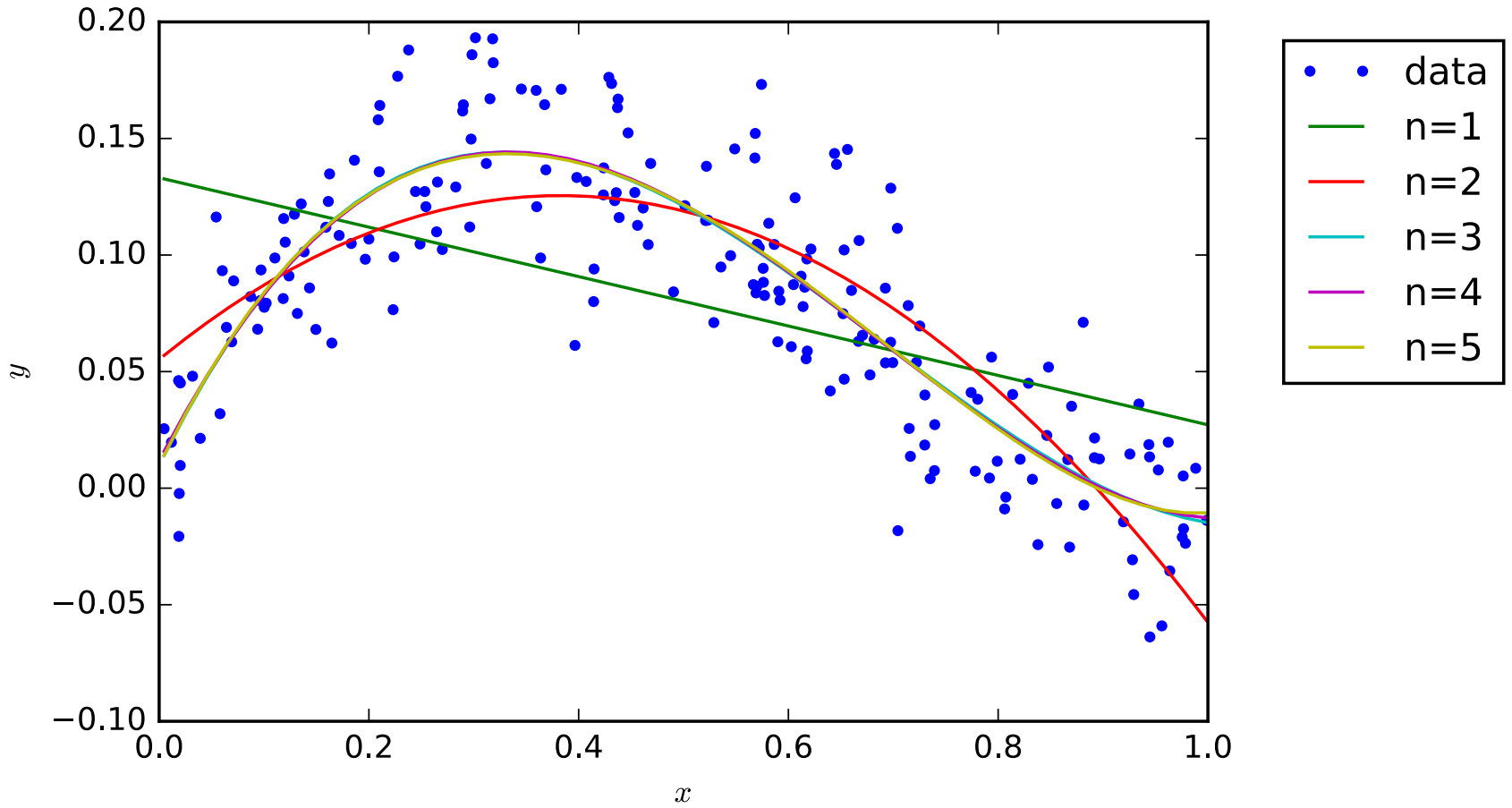
Underfitting

```
np.random.seed(0)  # Sets the random seed.
num_train = 200      # Number of training data points

# Generate the training data
x = np.random.uniform(low=0, high=1, size=(num_train,))
y = x - 2*x**2 + x**3 + np.random.normal(loc=0, scale=0.03, size=(num_train,))
f = plt.figure()
ax = f.gca()
ax.plot(x, y, '.')
ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
f.savefig('ml-basics_underfitting-data.pdf')
```



Underfitting



How do we think of overfitting and underfitting?

Overfitting and underfitting are important problems to address in machine learning.

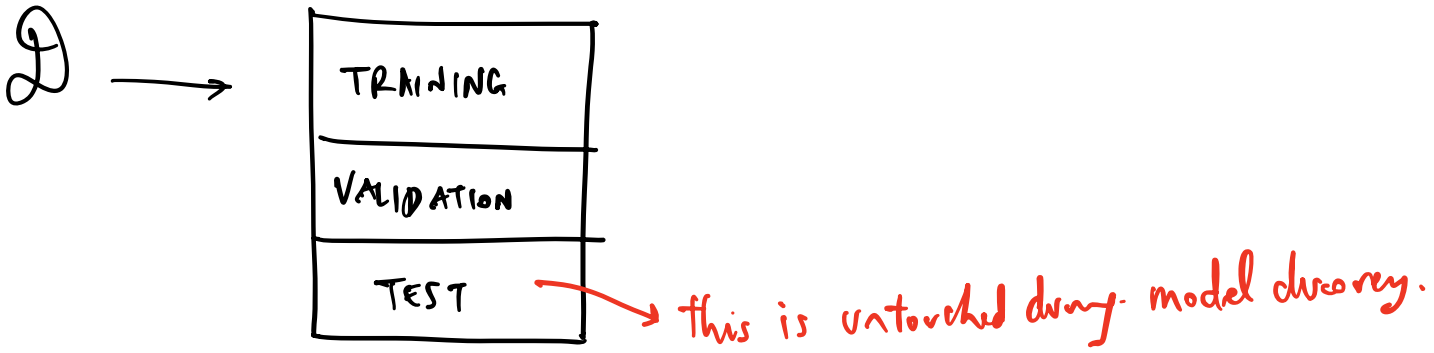
A more rigorous framework to think of these is as the *bias* and *variance* of the estimate of our parameters.

There is a very good theory / statistical method to choose appropriate models

Training, validation, and testing data

The standard for training, evaluating, and choosing models is to use different datasets for each step.

- **Training data** is data that is used to learn the parameters of your model.
- **Validation data** is data that is used to optimize the hyperparameters of your model. This avoids the potential of overfitting to nuances in the testing dataset.
- **Testing data** is data that is used to score your model.



Picking a best model

From this picture, a reasonable way to pick a model is to assess its generalization error, and pick a setting of the parameters that results in minimal value.

A brief aside:

Sometimes there may be scenarios where dataset size is so limited that it is not possible to have a reasonably good test dataset. In these cases, we may utilize *model selection*, which is itself an entire field.

At a high level, the principles of model selection is to find a way to *penalize* the model for being overly complex. There is an art to designing some penalty terms.

Criterion for selecting models include:

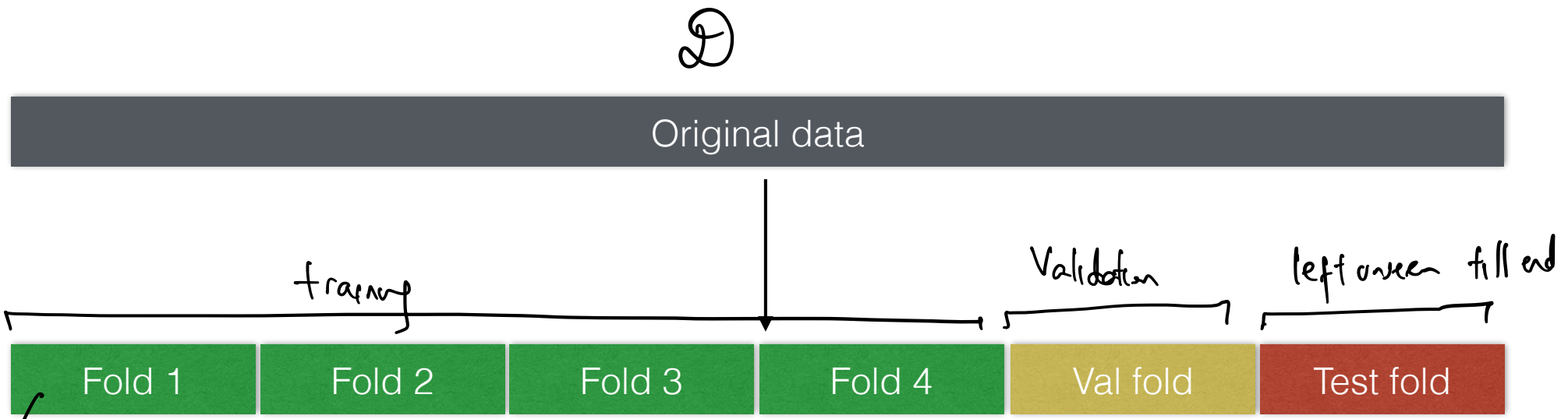
- Bayes information criterion
- Akaike information criterion
- Deviance information criterion

Look at other courses for ideas on model selection

We will really do only practical methods

My recommendation: if you have enough data to make a test set, do it.

Evaluating generalization error



- Note that in competitions, they will usually withhold a test dataset that you never will gain access to, but you can evaluate your model on it (a limited number of times per day).
  - In these cases, you should use all your original data for training (green) and validation (yellow).

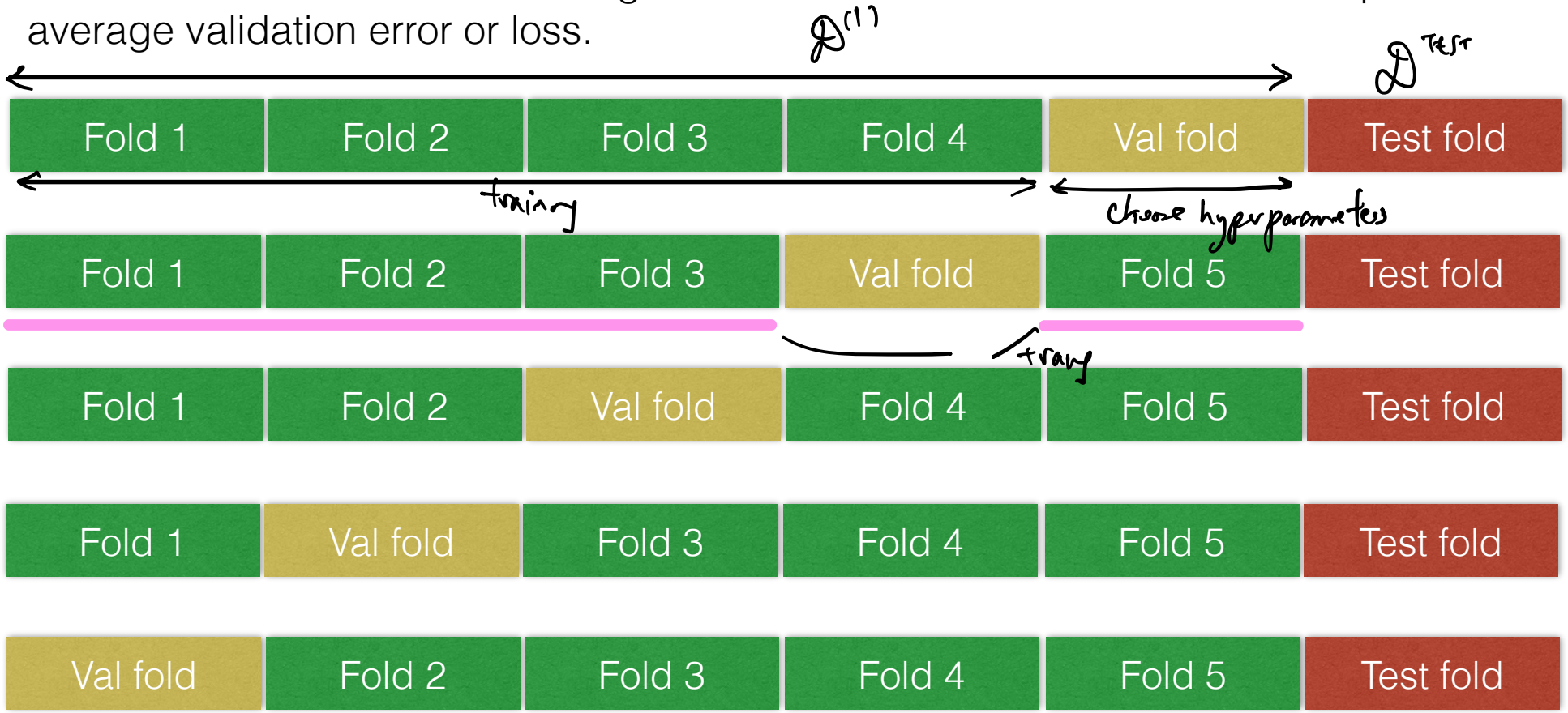
Chunks sizes are designed

Typically and in HWs the fold sizes are equal

$\mathcal{D} = [\ \mathcal{D}^{(1)} \quad \mathcal{D}^{\text{TEST}}\ ]$

Evaluating generalization error

In k-fold cross validation, split the data into k folds, and we train on the training data (green, folds 1 to k-1) and evaluate our model with this hyperparameter setting on the validation fold.You can also change which fold is the validation fold to compute an average validation error or loss.



through this we have created 5 data sets 5-fold cross-validation

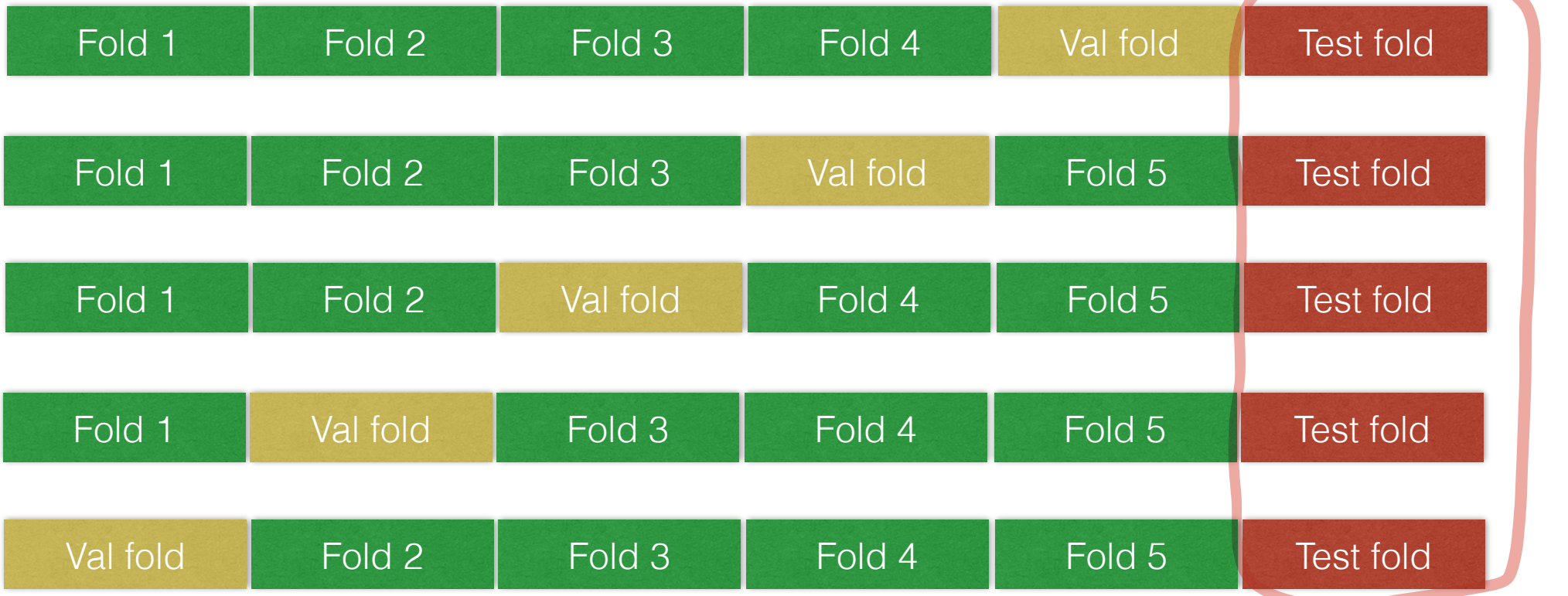
Train 5 models with each datasplit used to train a model

Evaluating generalization error

run 5-models with polynomial order 1
run 5-models with polynomial order 2
etc

In k-fold cross validation, split the data into k folds, and we train on the training data (green, folds 1 to k-1) and evaluate our model with this hyperparameter setting on the validation fold.You can also change which fold is the validation fold to compute an average validation error or loss.

NOT TOUCHED TILL FINAL MODEL IS OBTAINED



You then select the hyperparameters based on what has the best validation fold performance, and then you can train once more now across all 5 folds. Then you evaluate once on your test fold to score your model.

Training, validation, and testing

Explicit example:

train → model (for given choice of hyperparameters)
then use validation data to check performance on chosen parameters

- The CIFAR-10 dataset we download will contain 60,000 images.
  - **Test set**: We will set aside 10,000 examples, never to be touched except one time at the end of training to **score our model.**
- The remaining 50,000 images can be used for training and validation.
  - In **5-fold cross validation:**
    - We will split our data into **5 folds, each with 10,000 images.**
    - We will **train on 4 folds (40,000 images)** and evaluate on **1 fold (10,000 images)**. We will do this 5 times, each time changing which fold is the validation data.
    - We will average the validation accuracies (5 of them) to determine the best hyperparameters.
    - We can train one more time across all 50,000 images using the best hyperparameters.
  - In practice, training 5 different models (i.e., iterating which fold is the validation fold) is quite expensive for neural networks, so we will just make one validation dataset.

50,000 data points
10,000/fold each model uses
40,000 train and
10,000 validation.

k-fold cross validation from housing example

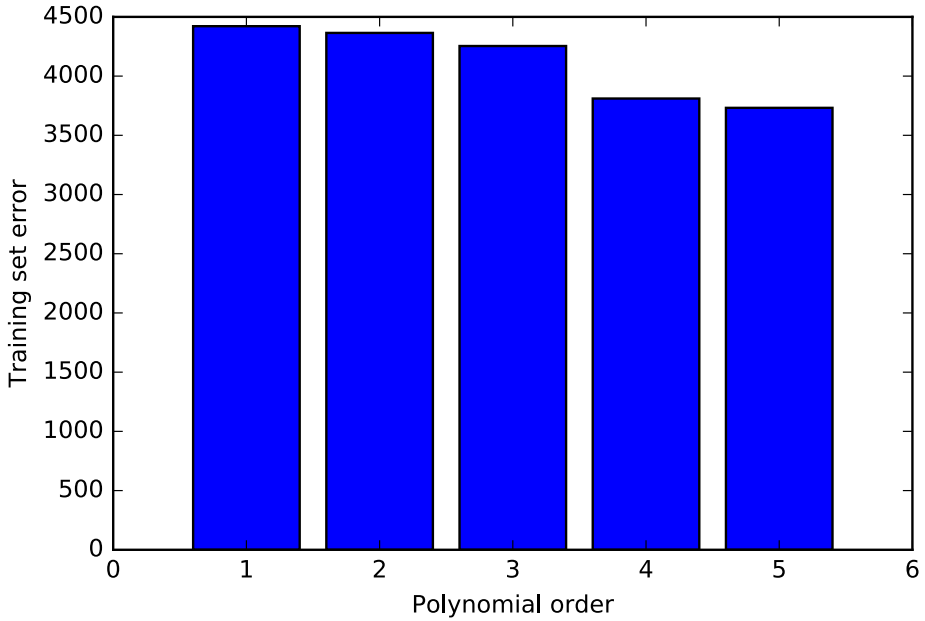
Routines for training models and testing data.

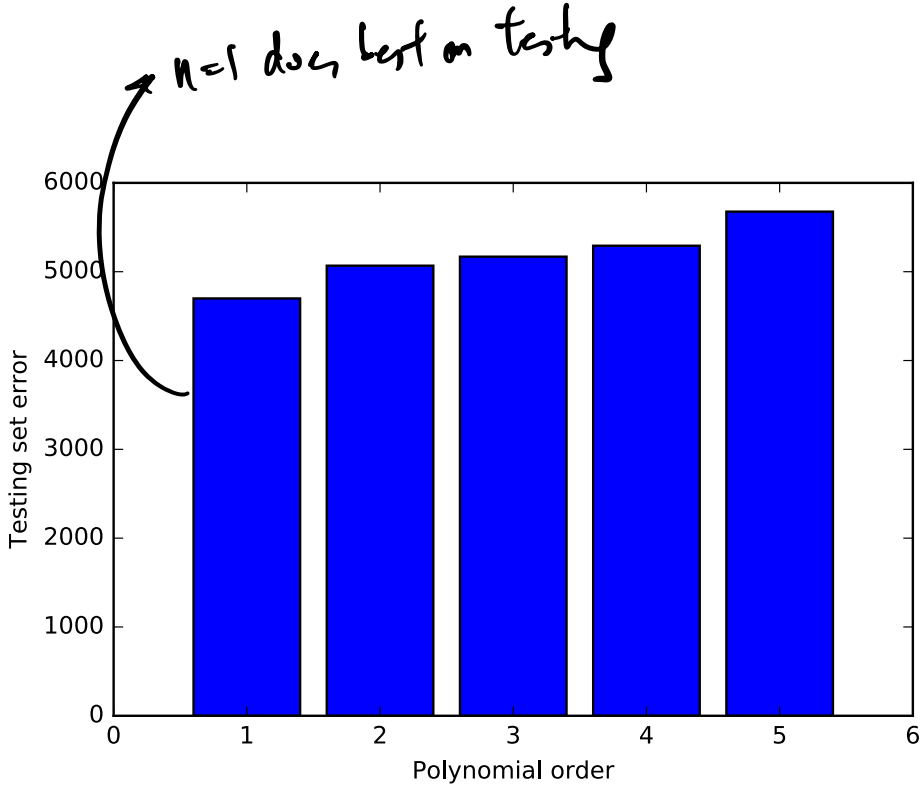
```
def train_models(x, y):
    # Fit a polynomial model up to N=5
    N = 5
    thetas = []
    for i in np.arange(N):
        xhat = np.vstack((x, np.ones_like(x))) if i == 0 else np.vstack((x**(i+1), xhat))
        thetas.append(np.linalg.inv(xhat.dot(xhat.T)).dot(xhat.dot(y)))

    return thetas

def test_models(x, y, thetas):
    N = 5
    errors = []
    for i in np.arange(N):
        xhat = np.vstack((x, np.ones_like(x))) if i == 0 else np.vstack((x**(i+1), xhat))
        errors.append(np.sqrt(np.sum((y - thetas[i].dot(xhat))**2) / len(y)))
    return errors
```

k-fold cross validation from housing example





Other types of optimization

continuous valued labels

We've talked about examples where we want to *minimize* a mean-square error or distance metric.

discrete labels what loss function to choose ?

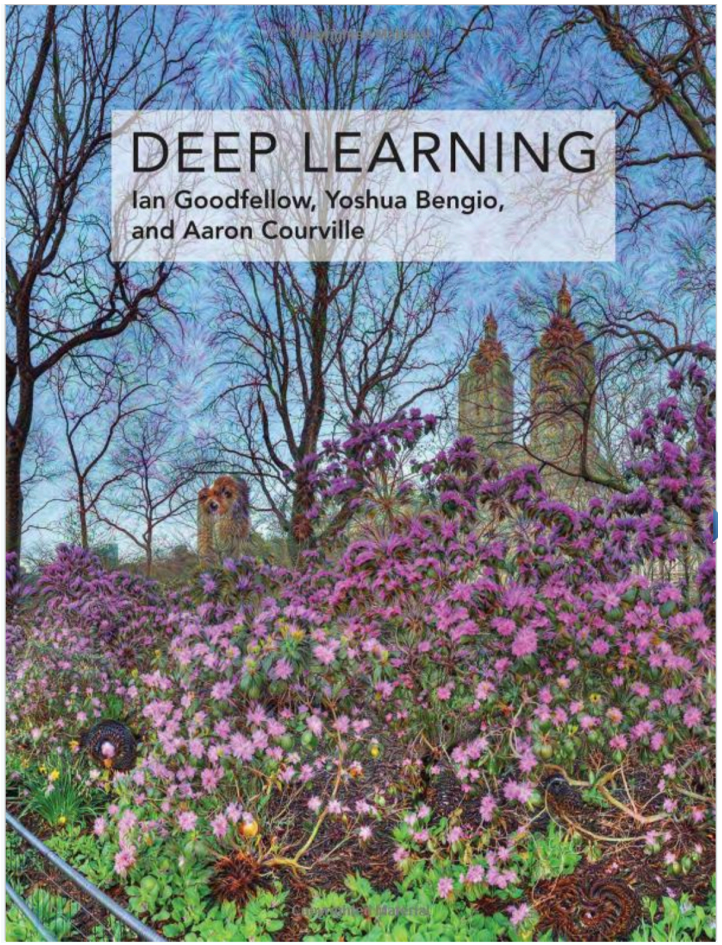
Another metric that we may want to minimize is the *probability of having observed the data*. In this framework, the data is modeled to have some distribution with parameters. We choose the parameters to maximize the probability of having observed our training data.

We will talk about this further when we discuss the "softmax classifier"

Supervised classification, maximum-likelihood, and gradient descent

Reading:

Deep Learning, 5.7, 5.9, 5.10, 5.11.1.



In this lecture, we'll cover some supervised learning techniques that are relevant for classification, and helpful for later lectures in neural networks.

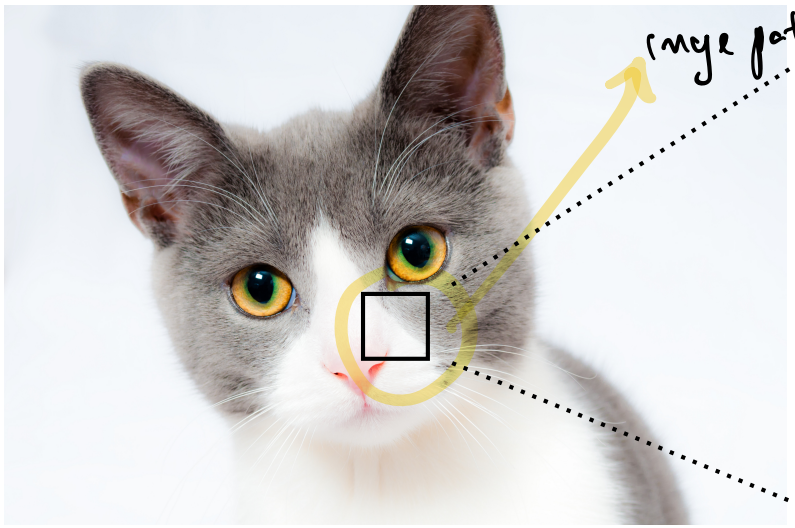
Image classification

This image is licensed under CC0. https://www.pexels.com/photo/grey-and-white-short-fur-cat-104827/

Image classification

[0, 255]
Color

Challenges:
- How to represent the data → eg. pixels for example in CIFAR-10 32 x 32 x 3 (pixel size of image)
- How to use this to do classification?



This image is licensed under CC0.

```
[[217, 215, 213, 213, 211, 210, 209, 207, 205, 203, 202, 201, 209],
 [218, 216, 215, 214, 211, 210, 209, 207, 205, 203, 202, 201, 209],
 [220, 218, 217, 216, 211, 210, 209, 207, 205, 203, 202, 201, 209],
 [220, 219, 217, 216, 211, 210, 209, 207, 205, 203, 202, 201, 209],
 [220, 220, 220, 220, 218, 217, 216, 214, 212, 210, 209, 208, 214],
 [220, 220, 220, 220, 218, 217, 216, 214, 212, 210, 209, 208, 214],
 [220, 220, 220, 220, 218, 217, 216, 214, 212, 210, 209, 208, 214],
 [220, 220, 220, 220, 218, 217, 216, 214, 212, 210, 209, 208, 214],
 [220, 220, 220, 220, 218, 217, 216, 214, 212, 210, 209, 208, 214],
 [220, 220, 220, 220, 218, 217, 216, 214, 212, 210, 209, 208, 214],
 [220, 220, 220, 220, 218, 217, 216, 214, 212, 210, 209, 208, 214],
 [220, 220, 220, 220, 214, 214, 212, 210, 209, 207, 205, 205, 214],
 [220, 220, 220, 220, 214, 214, 212, 210, 209, 207, 205, 205, 214],
 [220, 220, 220, 220, 214, 214, 212, 210, 209, 207, 205, 205, 214],
 [220, 220, 220, 220, 214, 214, 212, 210, 209, 207, 205, 205, 214],
 [220, 220, 220, 220, 214, 214, 212, 210, 209, 207, 205, 205, 214],
 [220, 220, 220, 220, 214, 214, 212, 210, 209, 207, 205, 205, 214],
 [220, 220, 220, 220, 214, 214, 212, 210, 209, 207, 205, 205, 214],
 [220, 220, 220, 220, 214, 214, 212, 210, 209, 207, 205, 205, 214]]
```

Images are stored in computers as a width x height x 3 array with numbers from [0, 255].

This is the input data for a computer vision algorithm.

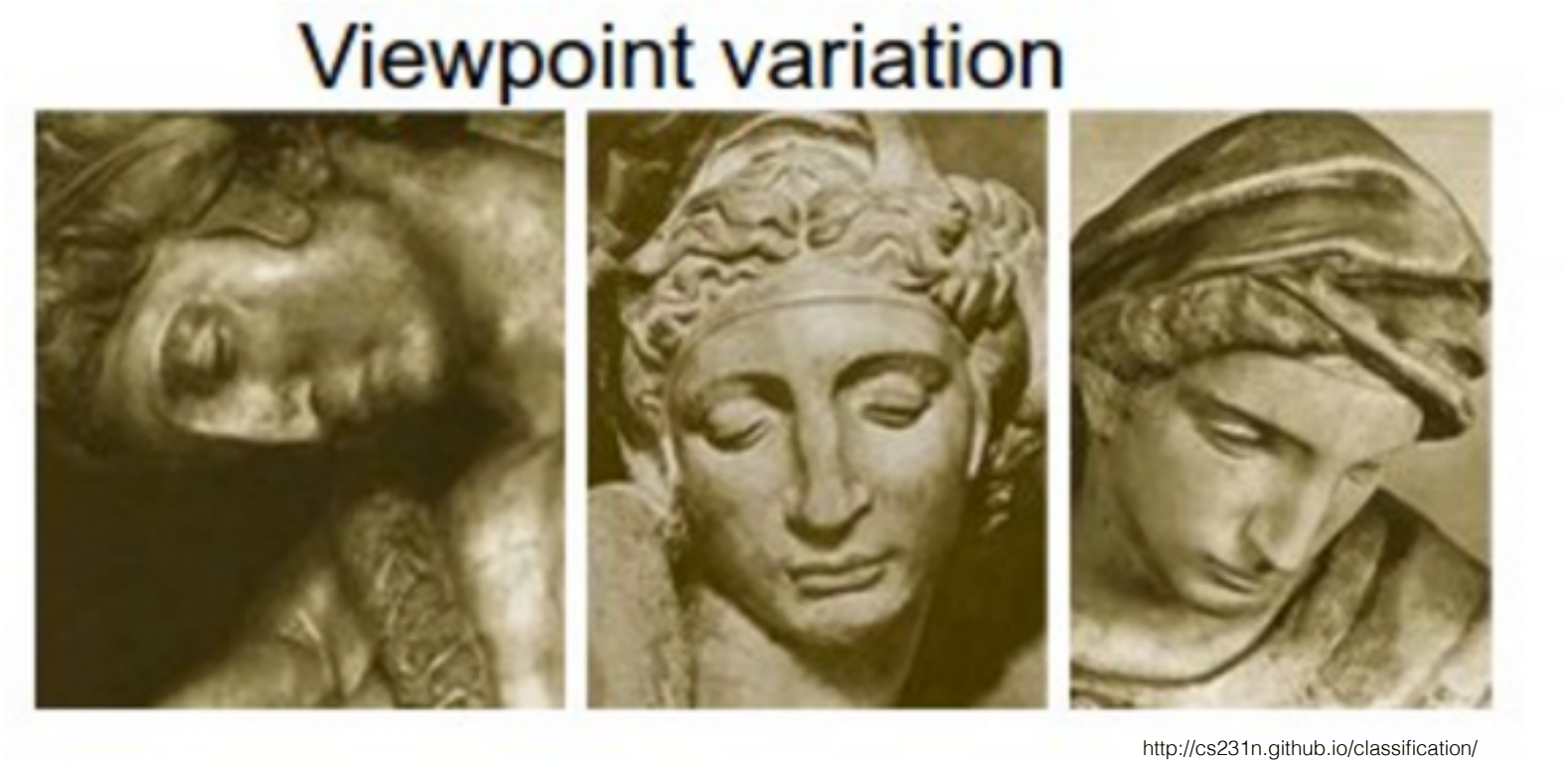
Image classification

As the image is stored as an array of numbers, there are several challenges associated with image classification.

- Viewpoint variations
- Zoom issue ie how magnified image is
- Postures / deformation
- Illumination
- Occlusion
- Background & blending
- Interclass variations

Image classification

Viewpoint variation:



http://cs231n.github.io/classification/

Image classification

Viewpoint variation

Zooming into or magnification

https://kevinzakka.github.io/2017/01/18/stn-part2/

Image classification

Illumination.

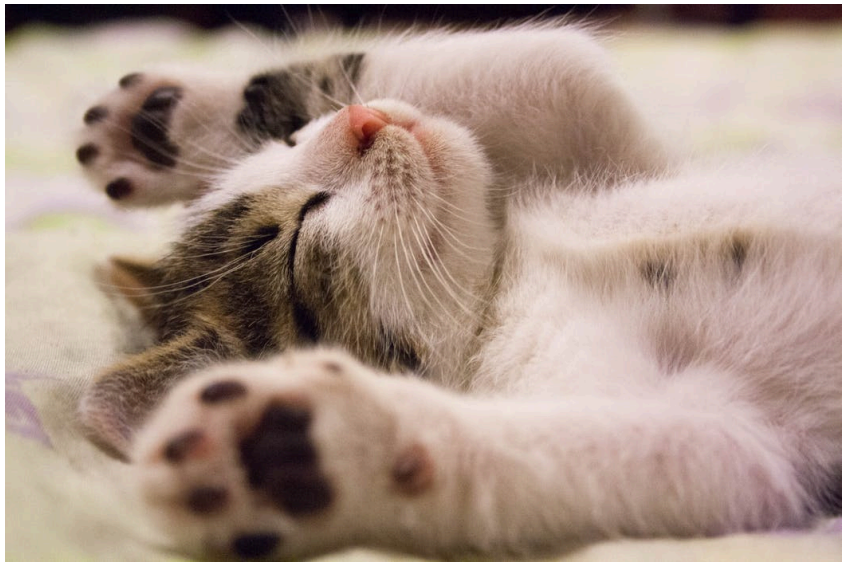
This image is licensed under CC0.

This image is licensed under CC0.

Image classification

Deformation.

This image is licensed under CC0.



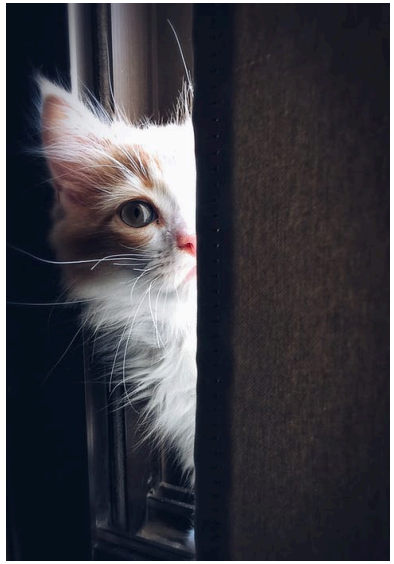
This image is licensed under CC0.

This image is licensed under CC0.

Image classification

Occlusion.

This image is licensed under CC0.



This image is licensed under CC0.

This image is licensed under CC0.

Image classification

Background clutter.

This image is licensed under CC0.

This image is licensed under CC0.

Image classification

Intraclass variation.

This image is licensed under CC0.

Classifying image data?

When trying to classify an image, there are several approaches one could think of taking.

In this class, we use the data driven approach, where we let a machine learning algorithm see a lot of data and learn a function mapping the image to class.

→ Features to focus on ⟶ earlier it was carefully designed

- NN paradigm ⟶ data driven

training data ⟶ model parameters and let features be learned through training

TEST ⟶ $\underline{\theta}$, model ⟶ Classification

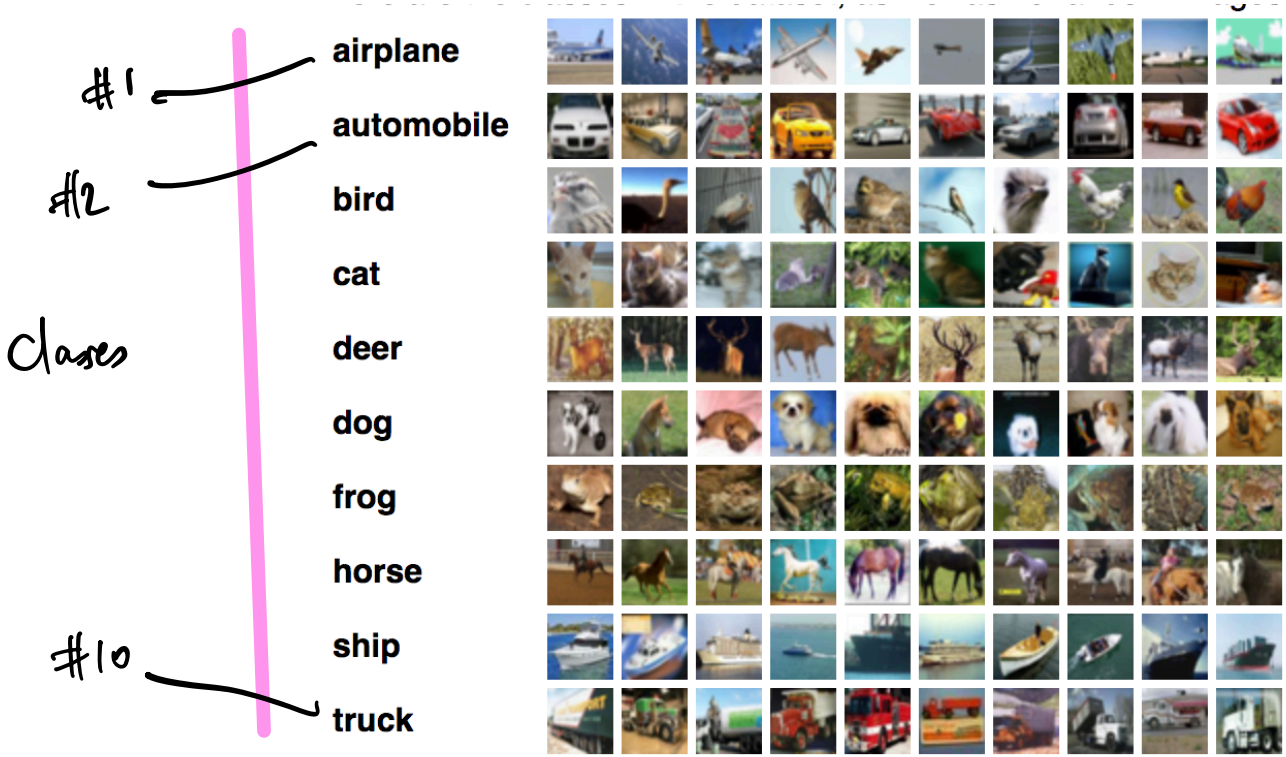
↑ test data

Classifying image data

For data, we will use the CIFAR-10 dataset: http://www.cs.toronto.edu/~kriz/cifar.html

The dataset comprises:

- 60,000 images that are 32 x 32 pixels in color (hence 3 channels).
- 10 classes, with 6,000 images per class.
- 50,000 images are for training and 10,000 are for testing.



Classifying image data

```
# Loading CIFAR-10
import os
import pickle

def load_CIFAR10(ROOT):
  """ load all of cifar """
  xs = []
  ys = []
  for b in range(1,6):
    f = os.path.join(ROOT, 'data_batch_%d' % (b, ))
    X, Y = load_CIFAR_batch(f)
    xs.append(X)
    ys.append(Y)
  Xtr = np.concatenate(xs)
  Ytr = np.concatenate(ys)
  del X, Y
  Xte, Yte = load_CIFAR_batch(os.path.join(ROOT, 'test_batch'))
  return Xtr, Ytr, Xte, Yte

def load_CIFAR_batch(filename):
  """ load single batch of cifar """
  with open(filename, 'rb') as f:
    datadict = pickle.load(f)
    X = datadict['data']
    Y = datadict['labels']
    X = X.reshape(10000, 3, 32, 32).transpose(0,2,3,1).astype("float")
    Y = np.array(Y)
    return X, Y
```

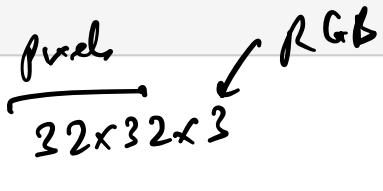
Acc: CS 231N from Stanford also used by Prof Icao.

Classifying image data

```
X_train, y_train, X_test, y_test = load_CIFAR10('cifar-10-batches-py')

print 'Training data shape: ', X_train.shape
print 'Training labels shape: ', y_train.shape
print 'Test data shape: ', X_test.shape
print 'Test labels shape: ', y_test.shape
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
```



We will reshape these 32x32x3 arrays into a 3072-dimensional vector.

These constitute the “input” data.

Classifying image data?

Supervised classification (cont.)

Consider a setup that is the same as the maximum-likelihood classifier of the previous lecture. Imagine you were given a training set of input vectors, $\{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(N)}\}$ and their corresponding classes, $\{y^{(1)}, y^{(2)}, \ldots, y^{(N)}\}$.

Now imagine that you were also given a new data point, $\mathbf{x}^{\text{new}}$. We found a way to classify through a probabilistic model, where we had to learn parameters, but isn't there a simpler way to classify that doesn't involve very much "machine learning" machinery?

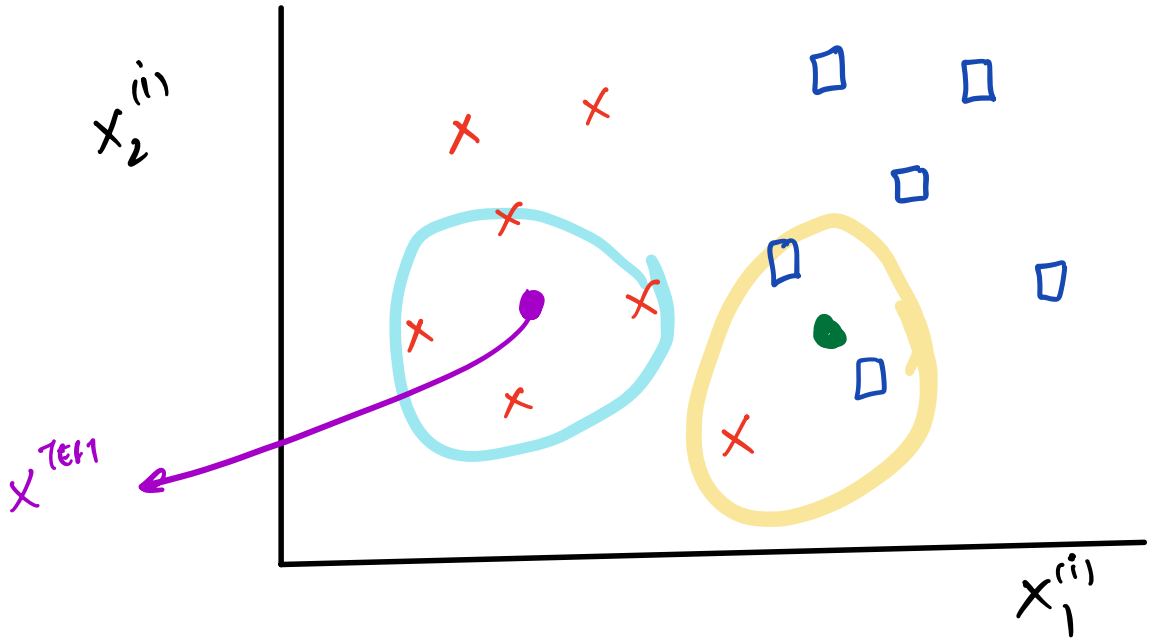
$$\mathcal{D} = \left\{ (\underline{x}^{(1)}, y^{(1)}), \ldots (\underline{x}^{(N)}, y^{(N)}) \right\}$$

TEST POINT x^{NEW} we want to find its label

k-nearest neighbors

$\underline{x}^{(i)} \in \mathbb{R}^{3072}$ $32 \times 32 \times 3$

Intuitively, k-nearest neighbors says to find the k closest points (or nearest neighbors) in the training set, according to an appropriate metric. Each of its k nearest neighbors then vote according to what class it is in, and $\mathbf{x}^{\text{new}}$ is assigned to be the class with the most votes.



× Class #1

□ Class #2

$\underline{x}^{(i)} \in \mathbb{R}^2$

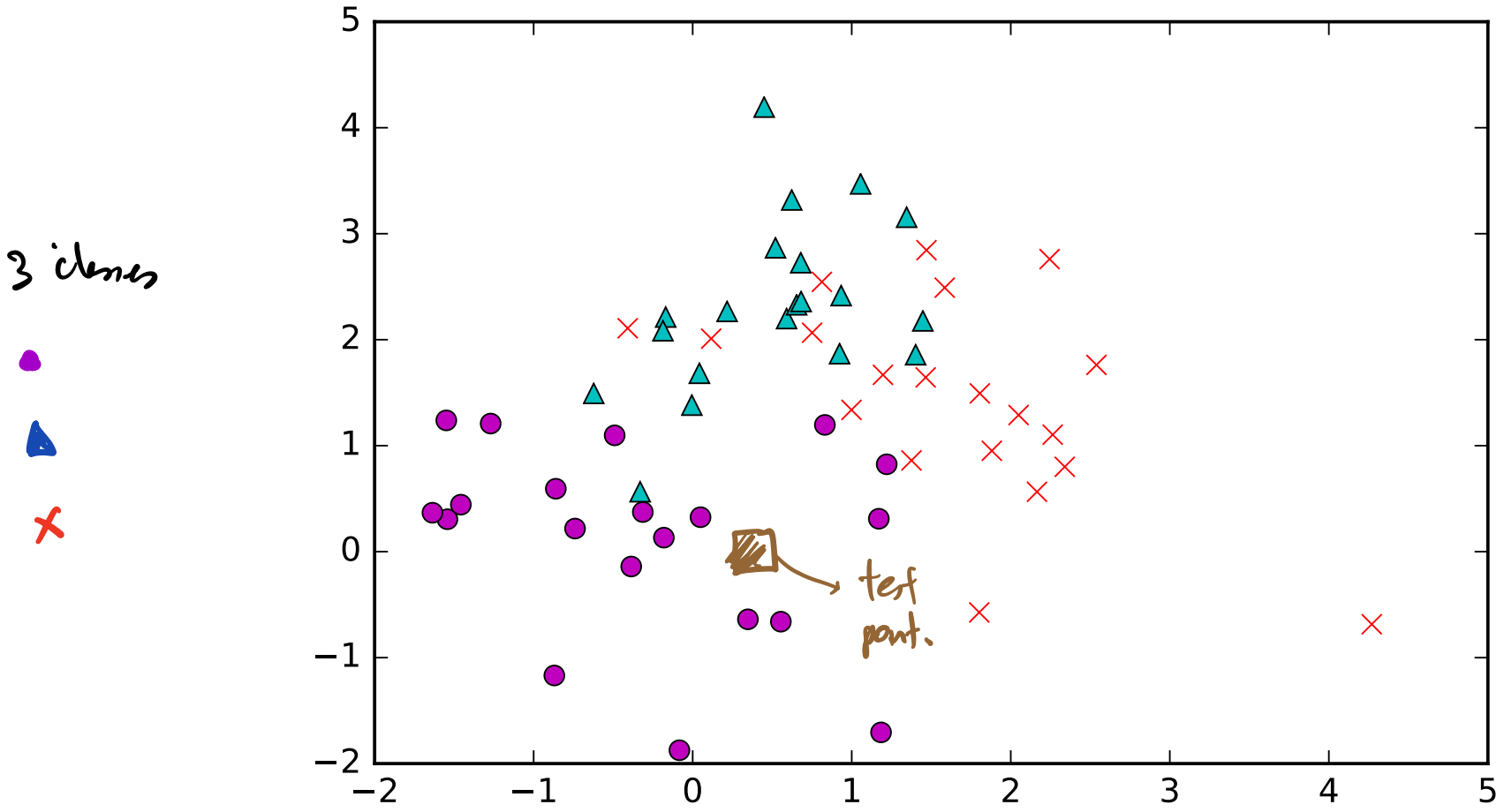
$y^{(i)} = \{\text{Red, blue}\}$

k-nearest neighbors, more formally

- Choose an appropriate distance metric, $d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$, returning the distance between $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(j)}$. E.g., $d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \left\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\right\|_2^2 = \sum_{k=1}^{n} |x_k^{(i)} - x_k^{(j)}|^2$
- Choose the number of nearest neighbors, k.
- Take desired test data point, $\mathbf{x}^{\text{new}}$, and calculate $d(\mathbf{x}^{\text{new}}, \mathbf{x}^{(i)})$ for $i = 1, \ldots, m$.
- With $\{c_1, \ldots, c_k\}$ denoting the k indices corresponding to the k smallest $d(\mathbf{x}^{\text{new}}, \mathbf{x}^{(i)})$, classify $\mathbf{x}^{\text{new}}$ as the class that occurs most frequently amongst $\{y^{c_1}, \ldots, y^{c_k}\}$. If there is a tie, any of the tying classes may be selected.

k-nearest neighbors

Example data:



k-nearest neighbors

How do we train the classifier? → The dataset is the model since we go back to dataset to find the nearest neighbor.

```
class KNearestNeighbor(object):

  def __init__(self):
    pass

  def train(self, X, y):
    self.X_train = X
    self.y_train = y
```

Pros? → Cost effective? → no training needed

Cons? →

- Memory needed would be large as you are keeping all the training data and not deriving a model
- Inference time compute
- Curse of dimensionality

k-nearest neighbors

How do we test a new data point?

```
class KNearestNeighbor(object):

  def __init__(self):
    pass

  def train(self, X, y):
    self.X_train = X
    self.y_train = y

  def test(self, x_test, k=1):

    dists = np.linalg.norm(self.X_train.T - x_test, axis=1).T
    sortedIdxs = np.argsort(dists)
    closest_y = self.y_train[sortedIdxs[:k]]
    y_pred = np.argmax(np.bincount(closest_y));

    return y_pred
```

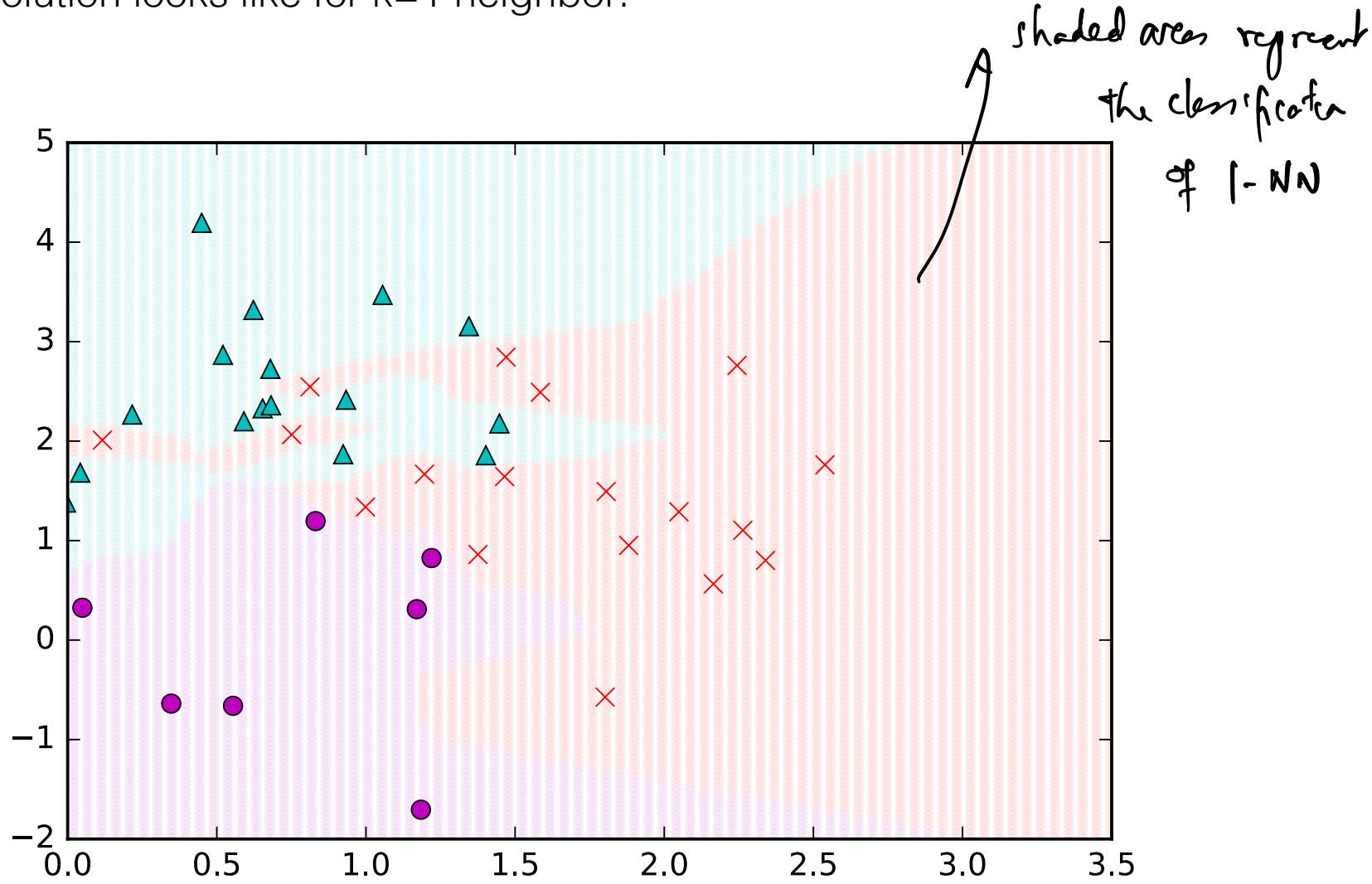
Pros?

Cons?

] See previous slide

k-nearest neighbors

What a solution looks like for k=1 neighbor:



k-nearest neighbors

What a solution looks like for k=3 neighbors:

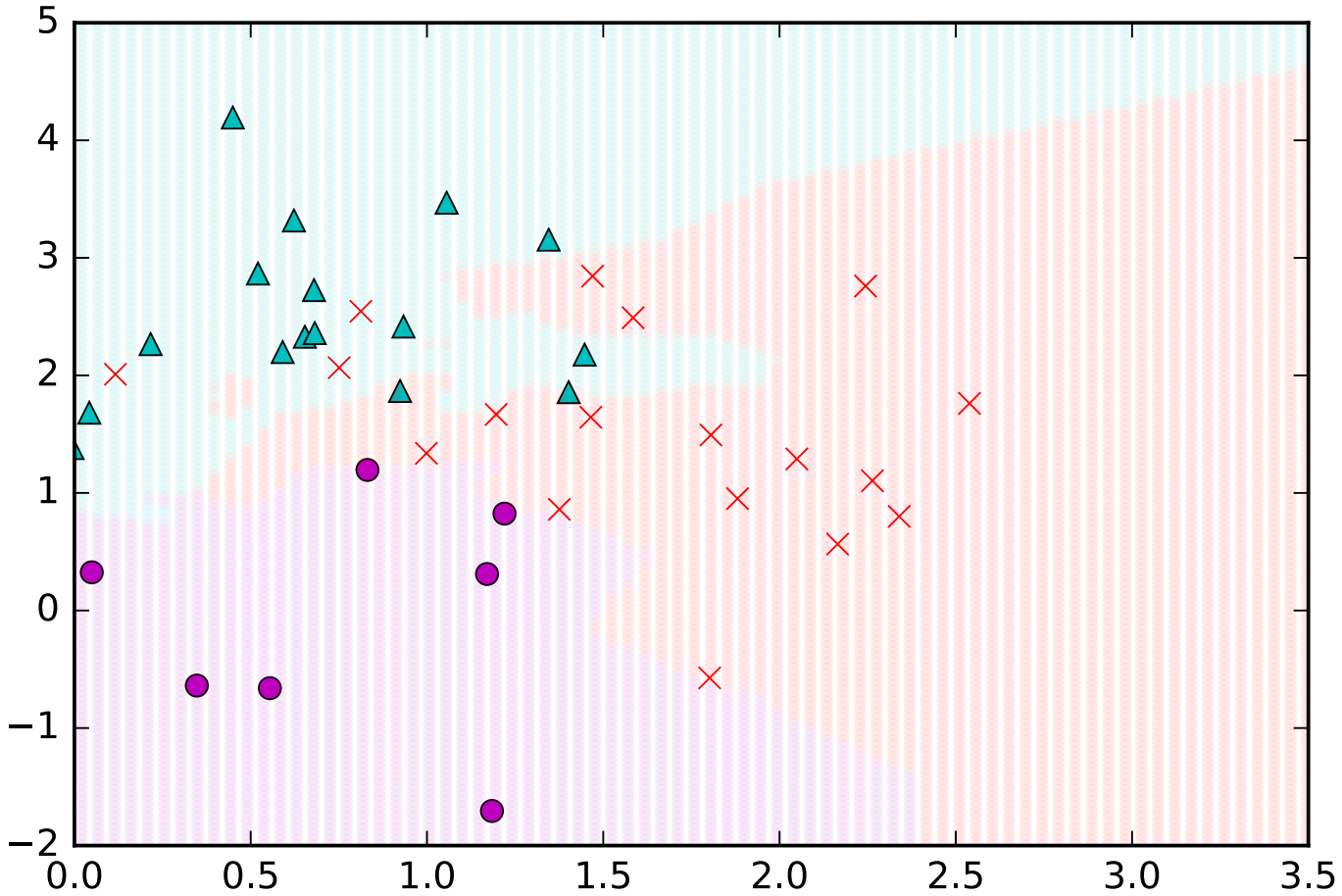
↳ outlier issue is addressed using layer k.

In modern AI inference is to be fast

k-NN inference is slow.

⇓

disadvantage / Con



k-nearest neighbors

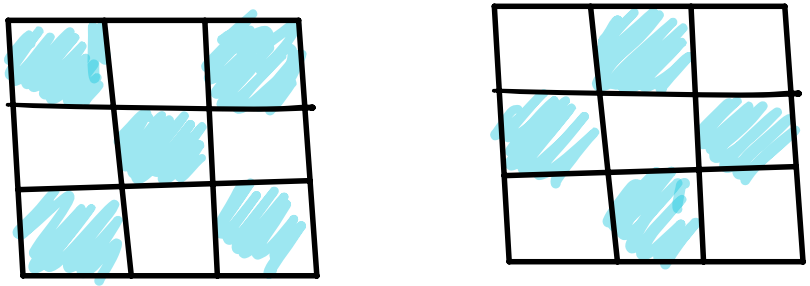
What are the hyperparameters of k-nearest neighbors?

- Distance metric choice
- k in k-NN

k-nearest neighbors

Why might k-nearest neighbors not be a good idea for image classification?

- Difficult to set a good distance metric



k-nearest neighbors

Why might k-nearest neighbors not be a good idea for image classification?

distance could be large

Original image is CC0 public domain

(all 3 images have same L2 distance to the one on the left)

Credit: CS231n, Stanford University

k-nearest neighbors

Why might k-nearest neighbors not be a good idea for image classification?

Curse of dimensionality:

- Images are very high-dimensional vectors, e.g., each CIFAR-10 image is a 3072 dimensional vector (and these are small images).

- Notions of "distance" become less intuitive in higher dimensions.
  - Distances in some dimensions matter more than others.
  - In higher-dimensional space, the volume increases exponentially.
  - This leaves a lot of empty space — and so the nearest neighbors may not be so near.

- Computationally could be harder

- Cover-Hart $\longrightarrow$ $Err_{NNC} \leq 2\, Err_{OPT} + \frac{2c\sqrt{d}}{N^{\frac{1}{1+d}}}$ d = dimension

  $\frac{2c\sqrt{d}}{N^{\frac{1}{1+d}}}$ → goes down very slowly with N

  N = dataset size

Classifiers based on linear classification

Perhaps a better way would be to develop a “score” for an image coming from each class, and then pick the class that achieves the highest score.

Linear classifiers are a major building block for neural networks. In particular, a common neural network layer (a multi-layer perceptron) comprises an affine transformation, followed by a nonlinear function.

The softmax classifier is the most common classifier at the end of a neural network.

Preview of how we work with Neural networks

- Output last layer is a "softmax" classifier ←
- Gets a "score" for image $x^{(i)}$ $\quad s(x^{(i)})$

$$z_i = \begin{bmatrix} s_1(x^{(i)}) \\ \vdots \\ s_c(x^{(i)}) \end{bmatrix}$$

Classifier picks the class with largest score

c = # of classes

$s_j(x^{(i)})$ score of class j on data $x^{(i)}$